Heart Attack Prediction Report

Prepared by: Ryan Stefanic
Email: ryan.stefanic1@gmail.com

Problem Statement:

In this project, I aimed to build a classification model for predicting whether a patient had heart disease or not, using various machine-learning techniques. Below is a detailed description of my approach, covering data pre-processing, feature engineering, model building and selection, hyperparameter tuning, performance evaluation, and lessons learned during the process.

Description:

The first thing I did was preprocess the data. When analyzing the data, I noticed that there were many zeros in the cholesterol column. These zeros were outliers and didn't make sense compared with the range of the rest of the data. To fix this issue, I replaced the zero values with the median cholesterol value of the other rows. The next thing I observed was that the dataset contained categorical variables that included both text and numerical data. To process the text values, I used one-hot encoding to turn the text into binary vectors, which allowed the model to effectively utilize the data. I finished the data cleaning by scaling the data and removing outliers.

I then started to build and test models, starting with random forest. I tested the models with the training set F1 score and cross-validation. I initially got decent results with the initial random forest model, with an F1 score around 0.85. I then tested both XGBoost and LightGBM and combined them with the random forest model to create a voting classifier. This voting classifier using these three models resulted in an F1 score of 0.89. This result made me believe that a voting classifier would be the best option for this project.

I then moved to hyperparameter tuning and feature engineering to improve the results. I used hyperparameter tuning for the three models, random forest, xgboost, and Lightgbm, in addition to other models that I added to the classifier, including SVM and gradient boosting. Every time I added a new model to the voting classifier or tuned the parameters, the F1 score slightly increased. I then engineered new features to capture more meaningful relationships in the data. New features I created included CholesterolPerAge, MaxHRPerAge, and RestingBPPerCholesterol. These features were created by dividing existing features, and as a result, I observed further improvements in cross-validation and F1 scores.

After hyperparameter tuning and feature engineering, I was stuck at an F1 score of around 0.8975. I tried creating more features, but nothing would give an improved score. I then tried to add more models to the voting classifier, but that did not improve the score either. Finally, changing the voting classifier from a hard vote, where each model casts a single vote for the predicted class, to a soft vote, where predictions are based on the average predicted probabilities, allowed me to reach an F1 score of 0.90196.

Lessons Learned:

Some unique ideas I learned during this project include the importance of understanding the data when creating new features. Many features I tried to add did not add anything of value to the model. When adding features, it's important to find new relationships between existing features that can help understand the target variable more. In addition, I learned that adding more and more models to a voting classifier comes with diminishing returns. Initially, each model added around 0.015 to the F1 score. Eventually, new models being added resulted in no improvement or the F1 score getting worse.